TfL "What-If" Simulation Database Design Document

1. Overview

This database models the London Underground network as a graph to support routing, disruption simulation and shortest-path analysis. Each station is represented as a node and connections between stations are represented as relationships.

A Neo4j graph database was chosen over a relational database because:

- The underground network is inherently graph shaped.
- Shortest-path and rerouting queries are more efficient and expressive.
- Relationships (connections) are first-class data, not join tables.

The database is used by the application to:

- Find the shortest route between two stations.
- Recalculate routes when stations or lines are disrupted.
- Perform future network analysis and simulations.

---

2. Conceptual Model

At a high level, the network consists of:

- Station nodes represent underground stations.
- CONNECTS_TO relationships represents direct track connections between stations

```
(Station) -[CONNECTS_TO]-> (Station)
```

Each connection stores metadata such as the line name and the distance between stations.

---

3. Node Definitions

3.1. Station Node

Description: Represents a single London Underground station.

Property	Type	Description
`id`	String	Unique TfL station identifier
`name`	String	Human-readable station name
`lat`	Float	Latitude coordinate
`lon`	Float	Longitude coordinate

4. Relationship Definitions

4.1. CONNECTS_TO Relationship

Description: Represents a direct connection between two consecutive stations on a tube line.

Although the underground is bidirectional, connections are stored as directed relationships to support traversal algorithms. Bidirectional travel is achieved by creating relationships in both directions where required.

Property	Type	Description
`line`	String	Line Identifier (e.g. `victoria`)
`lineName`	String	Human-readable line name
`distance`	Float	Estimated distance in meters

---

5. Constraints and Data Integrity

5.1. Station ID Uniqueness Constraint

```
CREATE CONSTRAINT station_id_unique IF NOT EXISTS
FOR(s:Station)
REQUIRE s.id IS UNIQUE;
```

Purpose:

- Prevents duplicate station nodes.
- Ensures consistent merging when importing data from multiple lines.
- Improves lookup performance by station ID.

This constraint guarantees that each real-world station exists exactly once in the graph.

---

6. Distance Calculation

Distances between stations are calculated using the Haversine formula, which estimates the straight-line distance between two points on the Earth's surface based on latitude and longitude.

Why this approach was chosen:

- TfL does not provide actual track lengths via the public API.
- Provides consistent edge weights for shortest-path algorithms.
- Computationally simple and reliable.

Limitations:

- Does not represent true tunnel or track length.

- Does not account for curvature of tracks or elevation.

Despite these limitations, haversine distance is sufficient for route comparison and disruption rerouting.

---

7. Data Ingestion Process

Data is sourced from the TfL Unified API, specifically:

- `/Line/{id}/Route/Sequence/all`

This endpoint provides ordered station sequences for each tube line. From this data:

- Stations are extracted and stored as nodes.
- Consecutive stations are connected via `CONNECTS_TO` relationships.
- Distances are calculated at ingestion time.

Duplicate stations across lines are handled using the unique station ID constraint.

---

8. Example Queries

8.1. Find a Station by Name

```
MATCH (s:Station {name: “Oxford Circus Underground Station”})
RETURN s;
```

Returns the Station node of Oxford Circus.

8.2. Find Shortest Path between Two Stations

```
MATCH (start:Station {name: "King's Cross St. Pancras Underground Station"})
MATCH (end:Station {name: "Waterloo Underground Station"})
CALL apoc.algo.dijkstra(start, end, 'CONNECTS_TO>', 'distance')
YIELD path, weight
RETURN path, weight;
```

This query finds the shortest route based on distance rather than number of stops:

- `path`: Neo4j path object containing nodes and relationships
- `weight`: sum of distance along the path.

When executed this query will display a graph showing the route that the path takes being:

- King’s Cross -> Russell Square -> Holborn -> Covent Garden -> Leicester Square -> Charing Cross -> Embankment -> Waterloo

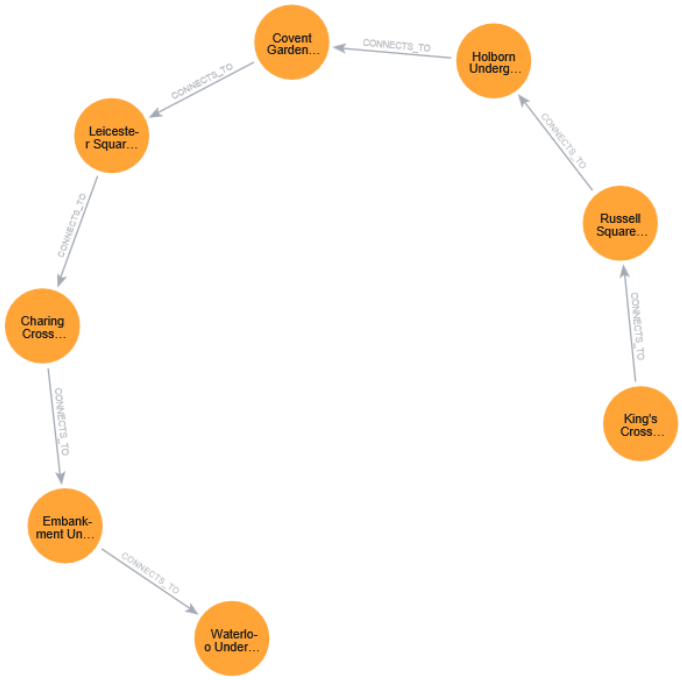

Figure 1 - Neo4j Graph of Dijkstra algorithm being performed from King's Cross to Waterloo station.

8.3. Find all stations from a specific line

```
MATCH (a:Station)-[r {line:"bakerloo"}]->(b:Station)
RETURN a, r, b
```

Returns all stations within a line, in this example it gets all the stations with the Bakerloo line.

---

9. Limitations

- Distances are estimated, not exact track lengths.
- Travel time and service frequency are not currently modelled.
- Real-time disruption data is not yet integrated.

---

10. Summary

This Neo4j database provides a clean, scalable representation of the London Underground network. By modelling stations and connections as a graph, it enables efficient shortest-path calculations and supports future expansion into more advanced transport simulations.